

NOTE DE DETTE TECHNIQUE

À traiter — sans urgence, mais sans oublier

ListTile sans Material

Des centaines d'avertissements qui noient les vraies erreurs

Deux widgets partagés en sont la cause



Nature Cosmétique en production, bloquant en développement
Origine Antérieure au chantier sauvegarde
Effort Environ deux heures
Priorité Basse, mais avant la prochaine campagne de débogage
Date 1^{er} septembre 2026

Ce qu'il faut savoir

Les journaux de l'application débitent en continu des blocs de ce genre :

```
1 EXCEPTION CAUGHT BY FLUTTER FRAMEWORK
2 The following assertion was thrown:
3 ListTile background color or ink splashes may be invisible.
4 The ListTile is wrapped in a DecoratedBox that has a background color.
5 Because ListTile paints its background and ink splashes on the nearest
6 Material ancestor, this DecoratedBox will hide those effects.
```

Pourquoi ça mérite d'être traité, même si « ça marche ». En production, l'effet est purement visuel : les ondes de clic ne se voient pas. Mais **en développement, chaque tuile fautive est remplacée par un bloc d'erreur rouge** et devient inutilisable. Et surtout, ces avertissements noient les vraies erreurs : une seule ouverture d'écran en produit plusieurs dizaines, au milieu desquelles une exception réelle passe inaperçue.

Vécu pendant le chantier sauvegarde. L'écran *Réglages* → *Sauvegarde* a été inutilisable plusieurs heures pour exactement cette raison, sans que personne ne comprenne pourquoi : les tuiles de fréquence et l'entrée de restauration étaient remplacées par des blocs d'erreur. Le diagnostic a coûté plusieurs allers-retours parce que le vrai message était perdu au milieu du bruit.

Table des matières

Ce qu'il faut savoir	1
1 La règle enfreinte	3
2 Où corriger	4
2.1 Deux widgets partagés produisent l'essentiel	4
2.2 Écrans à vérifier ensuite	4
3 Méthode et vérification	6
3.1 Repérer	6
3.2 Vérifier	6
3.3 L'ordre à suivre	6

CHAPITRE 1

La règle enfreinte

Un `ListTile` peint son fond et ses ondes de clic sur le **Material le plus proche** dans l'arbre. Si un widget intermédiaire porte lui-même une couleur de fond — `Container(color:)`, `DecoratedBox`, `ColoredBox` — ce fond masquerait ces effets, et Flutter lève une assertion plutôt que de laisser passer un rendu silencieusement faux.

Fautif	Correct
<code>Container(color: c, child: ListTile(...))</code>	<code>Material(color: c, child: ListTile(...))</code>
<code>Container(decoration: BoxDecoration(color: c, borderRadius: r))</code>	<code>Material(color: c, borderRadius: r, clipBehavior: Clip.antiAlias)</code>

`Material` accepte `color`, `borderRadius`, `elevation` et `shadowColor` : il remplace un `Container` décoratif sans rien perdre, et il est *plus* correct puisque c'est lui que les widgets `Material` attendent.

Ombres. Une ombre portée passe par `elevation` et `shadowColor` plutôt que par `boxShadow`. Le rendu diffère légèrement — l'élévation `Material` suit sa propre courbe — et c'est l'occasion d'uniformiser plutôt que de reproduire à l'identique.

CHAPITRE 2

Où corriger

2.1 Deux widgets partagés produisent l'essentiel

C'est la bonne nouvelle : il ne s'agit pas de reprendre trente écrans.

Fichier	Ce qui cloche
<code>lib/widgets/profile/settings_group.dart</code>	Le <code>Container(color: context.colors.surface)</code> qui enveloppe le contenu du groupe (ligne 31 environ). Il entoure tous les <code>SettingsNavTile</code> et <code>SwitchListTile</code> de tous les écrans de réglages. Le remplacer par un <code>Material</code> corrige l'essentiel des avertissements d'un seul geste.
<code>lib/screens/profile/profile_screen.dart</code>	La carte à <code>BoxDecoration(color: surface, boxShadow: AppShadows.subtle)</code> (ligne 280 environ) qui contient la colonne de <code>ListTile</code> — listes de contacts, trajets, déconnexion. C'est elle qu'on voit dans les journaux sous <code>_openContactLists</code> , <code>_openTrips</code> , <code>_logout</code> .

2.2 Écrans à vérifier ensuite

Ces fichiers contiennent à la fois un `ListTile` et un fond coloré. Tous ne sont pas fautifs — la coexistence ne suffit pas, encore faut-il que le fond soit un *ancêtre* de la tuile — mais c'est la liste à parcourir une fois les deux widgets partagés corrigés.

```
1 lib/screens/profile/preferred_contacts_screen.dart
2 lib/screens/profile/storage_screen.dart
3 lib/screens/profile/edit_profile_screen.dart
4 lib/screens/profile/qrcode_screen.dart
5 lib/screens/profile/contact_lists_screen.dart
6 lib/screens/profile/list_ringtones_screen.dart
7 lib/screens/profile/notification_settings_screen.dart
8 lib/screens/profile/account_security_screen.dart
9 lib/screens/chats/chats_screen.dart
10 lib/screens/chats/forward_message_screen.dart
11 lib/screens/chats/share_to_conversation_screen.dart
12 lib/screens/chats/conversation_media_screen.dart
13 lib/screens/chats/album_media_list_screen.dart
14 lib/screens/calls/keypad_screen.dart
15 lib/screens/calls/group_participants_picker_screen.dart
16 lib/screens/meetings/meets_screen.dart
17 lib/screens/status/status_create_screen.dart
```

```
18 lib/screens/onboarding/steps/profile_step.dart
```

Déjà corrigé. `lib/screens/profile/backup_screen.dart` l'a été pendant le chantier sauvegarde. Sa fonction `_card()` montre la forme à reproduire : `Padding` extérieur, `Material` avec `color`, `borderRadius` et `clipBehavior`, puis `Padding` intérieur.

CHAPITRE 3

Méthode et vérification

3.1 Repérer

```
1 # Fichiers combinant une tuile et un fond colore
2 for f in $(grep -rl "ListTile" lib/screens lib/widgets
3   --include=*.dart); do
4   grep -q "decoration: BoxDecoration\|ColoredBox\|Container(" "$f" &&
5     echo "$f"
6 done
```

3.2 Vérifier

Le compteur d'avertissements est la mesure la plus honnête. Avant correction, il se compte en centaines par ouverture d'écran :

```
1 adb logcat -c
2 # ... parcourir l'application : profil, réglages, discussions ...
3 adb logcat -d -s flutter:V | grep -c "ink splashes may be invisible"
```

L'objectif est **zéro**. Un compteur qui descend sans atteindre zéro signale qu'un écran de la liste n'a pas été traité.

3.3 L'ordre à suivre

1. `settings_group.dart` — le plus rentable, il couvre tous les écrans de réglages d'un coup.
2. `profile_screen.dart` — la deuxième source la plus bruyante.
3. Mesurer. Le reste de la liste ne mérite d'être parcouru que si le compteur n'est pas déjà à zéro.

À faire avant la prochaine campagne de débogage. Ce n'est pas urgent en soi. Mais tant que ces avertissements coulent, **toute recherche de bug dans l'application coûte plus cher qu'elle ne devrait**, et certains défauts resteront invisibles. Le traiter est un investissement sur la capacité à diagnostiquer, pas une correction cosmétique.

Fin de la note.